

T0.3 — Meta webhook signature obligatorio

Diagnostico de solo lectura · Fecha: 2026-05-01 · Modo: solo lectura. · Sin cambios, sin commits, sin push.

Hallazgo separado durante este diagnostico: agent/providers/whapi.py no implementa validacion de firma. Si WHATSAPP_PROVIDER=whapi en produccion (default del repo), **existe un P0 abierto independiente de T0.3** que se anota como T0.10 nueva al final.

1. Flujo actual del webhook de Meta/WhatsApp

```
WhatsApp/Meta servers -> POST https:///webhook
|
agent/main.py:2154  webhook_handler(request)
|
agent/main.py:771   procesar_webhook(request)
|
proveedor.parsear_webhook(request)
|
+-- WHATSAPP_PROVIDER=whapi   -> ProveedorWhapi.parsear_webhook
+-- WHATSAPP_PROVIDER=meta    -> ProveedorMeta.parsear_webhook
+-- WHATSAPP_PROVIDER=twilio  -> ProveedorTwilio (no presente)
|
(Solo si Meta) ProveedorMeta._verificar_firma(body, X-Hub-Signature-256)
|
+-- firma valida   -> parsea con Pydantic, retorna mensajes
+-- firma invalida -> retorna []
+-- sin app_secret -> return True (acepta sin verificar)
|
Mensajes entrantes -> procesados por brain.py / Claude
```

Verificacion adicional ya presente (agent/providers/meta.py:191-207): replay protection con ventana de 600 s sobre timestamp del mensaje. **Pero vive despues de la firma:** si un atacante forja un payload sin firma, la replay protection no lo salva.

2. Endpoint receiver

agent/main.py:

Endpoint	Linea	Funcion
GET /webhook	341	webhook_verificacion -> proveedor.validar_webhook (challenge Meta para subscription)
POST /webhook	2154	webhook_handler -> procesar_webhook -> proveedor.parsear_webhook
POST /webhook/messages	2160	mismo handler que /webhook (alias para Whapi cuando configura /messages suffix)

El endpoint POST /webhook es unico. Que parser corre depende de la env var WHATSAPP_PROVIDER. No hay rutas separadas por provider.

3. Como se valida hoy la firma de Meta

agent/providers/meta.py:125-150:

```
def _verificar_firma(self, body_bytes: bytes, signature_header: str) -> bool:
    if not self.app_secret:
```

```
logger.warning("[META] META_APP_SECRET no configurado – webhook sin verificacion HMAC...")
return True # Permitir sin firma si no se configuro (backwards compatible)

if not signature_header or not signature_header.startswith("sha256="):
    logger.warning("[META] Webhook recibido sin firma X-Hub-Signature-256 valida – rechazado")
    return False

expected = hmac.new(self.app_secret.encode(), body_bytes, hashlib.sha256).hexdigest()
if not hmac.compare_digest(signature_header[7:], expected):
    logger.warning("[META] Firma HMAC del webhook NO coincide – payload rechazado")
    return False

return True
```

Buenas practicas que ya hace bien:

- `hmac.compare_digest` (timing-safe).
- Validacion del prefijo 'sha256='.
- HMAC-SHA256 con `app_secret` como key y body crudo como mensaje.

Problema: la primera condicion (`if not self.app_secret: return True`) es el **fallback inseguro** — el patron explicito que cierra T0.3.

4. Que pasa si META_APP_SECRET no esta configurado?

Camino	Comportamiento
Provider activo es Meta + META_APP_SECRET vacio	Acepta cualquier payload sin firma. Solo emite warning. Procesa los mensajes como reales.
Provider activo es Meta + META_APP_SECRET seteado + firma invalida	Rechaza con False -> parsear_webhook retorna [] -> endpoint responde 200 OK sin procesar.
Provider activo es Meta + sin header X-Hub-Signature-256	Rechaza con False (mismo path).
Provider activo es Whapi	_verificar_firma ni se invoca (Whapi no la implementa — ver §10.3).

enhanced/diagnostics.py:135-144 tambien lee META_APP_SECRET pero solo para reportar diagnostico al admin, no para validar.

Tests existentes (tests/test_providers.py:237-285) codifican el comportamiento permisivo:

- test_sin_secret_permite_todo (linea 238) — asume y verifica el path inseguro.
- test_firma_valida / test_firma_invalida / test_sin_header_firma cubren el camino seguro.
- test_webhook_rechaza_firma_invalida / test_webhook_acepta_firma_valida E2E.

Hay que **actualizar** test_sin_secret_permite_todo para que solo aplique en dev/test. Los demas siguen igual.

5. El endpoint publico permite mensajes falsos o payloads no firmados?

Depende del provider activo en produccion.

Provider activo	Endpoint publico	Acepta payloads no firmados?
Meta y META_APP_SECRET no seteado	si	SI — riesgo P0 abierto
Meta y META_APP_SECRET seteado	si	No, rechaza con 200+[]
Whapi (default actual)	si	SI — riesgo P0 separado, ver §10.3

Necesito tu confirmacion operativa: cual es el valor de WHATSAPP_PROVIDER en Render? Eso decide si T0.3 cierra un riesgo activo (Meta) o solo blinda codigo preparado para el futuro (igual que paso con Stripe T0.2). El repo apunta a whapi como default, pero el deploy puede tener otra configuracion.

6. Variables que deben estar en Render antes del merge

Variable	Origen	Formato	Cuando es obligatoria
META_APP_SECRET	Meta for Developers -> tu App -> 'Settings' -> 'Basic' -> 'App Secret' -> 'Show'	string hex de 32 chars (sin prefijo)	Solo si WHATSAPP_PROVIDER=meta en Render
WHATSAPP_PROVIDER	env var existente	whapi meta twilio	siempre

No solicito ver el valor. Solo confirmacion de que esta seteado en Render **si** el provider activo es Meta.

Si el provider activo es Whapi, el check de Meta solo se ejecutaria si alguien cambiara WHATSAPP_PROVIDER=meta sin setear META_APP_SECRET. T0.3 protege ese caso futuro y cierra la superficie de ataque del codigo (similar a T0.2 backend).

7. Cambio minimo recomendado

Patron distinto al de T0.2. En Stripe el check se hizo al import del modulo. En Meta no es ideal porque:

- El app_secret se lee en ProveedorMeta.__init__(), no a nivel modulo.
- agent/providers/__init__.py:obtener_proveedor() solo importa Meta si WHATSAPP_PROVIDER=meta. Si esta en Whapi, Meta no se carga al startup.
- Hacer el check al import del modulo bloquearia tests y dev runs aunque se elija otro provider.

Recomendacion: hacer el check en ProveedorMeta.__init__():

```
def __init__(self):
    self.access_token = os.getenv("META_ACCESS_TOKEN", "")
    self.phone_number_id = os.getenv("META_PHONE_NUMBER_ID", "")
    self.verify_token = os.getenv("META_WEBHOOK_VERIFY_TOKEN", "dona_webhook_secret")
    self.app_secret = os.getenv("META_APP_SECRET", "").strip()
    self.api_version = os.getenv("META_API_VERSION", "v21.0")
    # ...
    environment = os.getenv("ENVIRONMENT", "development").lower()
    if environment == "production" and not self.app_secret:
        raise RuntimeError(
            "[META] META_APP_SECRET no configurado en produccion - "
            "webhooks aceptarían payloads forjados, lo que permite a un "
            "atacante inyectar mensajes WhatsApp falsos. Configura la "
            "variable o cambia WHATSAPP_PROVIDER antes del deploy."
        )
    )
```

Y en _verificar_firma — defensa en profundidad:

```
def _verificar_firma(self, body_bytes: bytes, signature_header: str) -> bool:
    environment = os.getenv("ENVIRONMENT", "development").lower()
    if not self.app_secret:
        if environment == "production":
            logger.error(
                "[META] META_APP_SECRET no configurado en produccion - "
                "rechazando webhook (defensa en profundidad)."
            )
            return False
        logger.warning(
            "[META] META_APP_SECRET no configurado - aceptando sin verificar "
            "(INSEGURO, solo dev/test)."
        )
        return True
    # ... resto del codigo actual sin cambios
```

Esto da **dos capas**:

- __init__ aborta el deploy si Meta es el provider activo en prod sin secret.
- _verificar_firma rechaza igual si por algun path el provider se instancia sin secret (defensa en profundidad).

Total de cambio

- agent/providers/meta.py: +12 lineas en __init__ + ~8 lineas modificadas en _verificar_firma.
- .env.example: +1 bloque documental para META_APP_SECRET.

- tests/test_providers.py: actualizar test_sin_secret_permite_todo y agregar TestVerificacionHMACProduction con 4 nuevos.
- **PR pequeno, ~25-35 lineas netas.** Mas chico que T0.2.

8. Archivos que tocaria

Archivo	Tipo de cambio
agent/providers/meta.py	Agregar check en ProveedorMeta.__init__ + agregar branch production en _verificar_firma.
.env.example	Documentar META_APP_SECRET (hoy aparece comentado en el bloque Meta general; explicitar que es obligatorio en produccion).
tests/test_providers.py	Renombrar/actualizar test_sin_secret_permite_todo -> version dev. Agregar TestVerificacionHMACProduction con casos production.

Lo que NO toco:

- agent/main.py — no requiere import top-level. El check vive en el __init__ del provider, que se ejecuta cuando obtener_proveedor() se llama en main.py:54.
- agent/providers/whapi.py — fuera del alcance de T0.3 (ver §10.3 sobre el riesgo separado).
- agent/providers/__init__.py (factory) — sin cambios.
- Endpoint webhook_verificacion (GET) — usa verify_token, no app_secret. No es parte de T0.3.

9. Tests a correr

9.1 Tests existentes a actualizar

tests/test_providers.py clase TestVerificacionHMAC (lineas 237-285):

- test_sin_secret_permite_todo (linea 238): renombrar a test_sin_secret_en_dev_permite y setear monkeypatch.setenv('ENVIRONMENT', 'development') explicito. Mantener el assert.
- test_firma_valida, test_firma_invalida, test_sin_header_firma: sin cambios.
- test_webhook_rechaza_firma_invalida, test_webhook_acepta_firma_valida: sin cambios.

9.2 Tests nuevos a agregar

```
class TestVerificacionHMACProduction:
    def test_production_sin_secret_levanta_runtime_error_en_init(self, monkeypatch):
        monkeypatch.setenv("ENVIRONMENT", "production")
        monkeypatch.delenv("META_APP_SECRET", raising=False)
        with pytest.raises(RuntimeError, match="META_APP_SECRET"):
            ProveedorMeta()

    def test_production_con_secret_no_levanta_en_init(self, monkeypatch):
        monkeypatch.setenv("ENVIRONMENT", "production")
        monkeypatch.setenv("META_APP_SECRET", "test_secret_dummy")
        ProveedorMeta() # No debe levantar.

    def test_production_runtime_sin_secret_rechaza_firma(self, monkeypatch):
        # Defensa en profundidad: aunque el __init__ se haya saltado, runtime rechaza.
        proveedor = ProveedorMeta()
        proveedor.app_secret = ""
```

```
monkeypatch.setenv("ENVIRONMENT", "production")
assert proveedor._verificar_firma(b"body", "sha256=anything") is False

def test_dev_sin_secret_permite_todo(self, monkeypatch):
    monkeypatch.setenv("ENVIRONMENT", "development")
    proveedor = ProveedorMeta()
    proveedor.app_secret = ""
    assert proveedor._verificar_firma(b"body", "") is True
```

Total: **1 actualizado + 4 nuevos**.

9.3 Suite completa

pytest -q. Estimado: 449 actuales + 4 nuevos = **~453 tests**, ~35 s.

10. Riesgos de compatibilidad y deploy

10.1 Riesgos del PR

Riesgo	Probabilidad	Mitigacion
WHATSAPP_PROVIDER=meta en Render sin META_APP_SECRET -> deploy aborta	Solo si meta activo	Confirmar antes del merge
WHATSAPP_PROVIDER=whapi en Render -> Meta no se instancia -> check no ejecuta -> deploy ok	Probable (default)	Sin accion; T0.3 igual blinda el codigo
META_APP_SECRET mal configurado (espacios, caracteres invisibles)	Baja	.strip() ya aplicado en el codigo propuesto
Tests existentes rompen	1 test cambia de contrato	Plan: actualizar (§9.1)
Provider Meta funcional rechaza webhook real por mismatch de secret	Solo si secret en Render no matchea con App Secret en Meta Dashboard	Verificar antes

10.2 Compatibilidad con WhatsApp/Meta

- X-Hub-Signature-256 es estandar Meta — sin cambios en como Meta firma.
- Replay protection (timestamps) sigue activa — sin cambios.
- Validacion GET de subscripcion (hub.challenge) sigue activa — sin cambios.
- META_WEBHOOK_VERIFY_TOKEN (otro secret distinto) — fuera de T0.3.

Rollback: si tras el merge un webhook real es rechazado por mismatch, revertir el commit en Render. El comportamiento previo (acepta sin firma) vuelve y los webhooks legitimos pasan.

10.3 Riesgo separado P0 — Whapi sin validacion de firma

Hallazgo nuevo durante este diagnostico: agent/providers/whapi.py **no implementa validacion de firma**. Whapi si soporta firma de webhooks (HMAC-SHA256 con secret configurado en panel Whapi), pero el codigo del repo simplemente hace request.json() y procesa el body crudo.

Implicacion: si WHATSAPP_PROVIDER=whapi en produccion (default), **cualquiera que conozca la URL /webhook puede enviar payloads forjados** y Dona los procesara como mensajes legitimos. Esto es un **P0 abierto independiente** que el plan v2.1 no listaba como tal.

Esto NO es alcance de T0.3. Es un **T0.10 nuevo** que deberia tomarse despues de T0.3 como Phase 0 extendida. Lo senalo aqui para que no quede oculto.

11. Checklist antes de mergear

11.1 Pre-PR (antes de empezar)

- [] Confirmar el valor actual de WHATSAPP_PROVIDER en Render. Si es meta, tambien confirmar META_APP_SECRET.

[] Si WHATSAPP_PROVIDER=meta: confirmar que el secret en Render coincide con el 'App Secret' de Meta for Developers (mismo string hex 32 chars).

[] Decidir nombre de branch: sugiero **pr/meta-webhook-firma-obligatoria**.

[] Decidir si T0.10 (whapi sin firma) entra al backlog Phase 0 explicitamente.

11.2 Durante el PR (local)

[] Implementar cambio en agent/providers/meta.py: __init__ y _verificar_firma.

[] Documentar META_APP_SECRET en .env.example.

[] Actualizar/extender tests en tests/test_providers.py.

[] Correr `pytest -q` localmente — 100% verde.

11.3 Pre-push

[] Verificar que el diff toca solo los 3 archivos previstos.

[] Verificar que no se filtro el secret real en codigo, comentarios ni tests.

[] Commit: *fix(seguridad): META_APP_SECRET obligatorio en produccion (T0.3)*.

11.4 Post-push, pre-merge

[] Push a origin/pr/meta-webhook-firma-obligatoria.

[] Abrir PR contra main en GitHub.

[] **NO mergear** hasta confirmar la matriz de env vars.

11.5 Post-merge

[] Render deploy debe iniciar y completar segun el provider activo:

- Si WHATSAPP_PROVIDER=whapi: deploy normal (Meta ni se instancia).
- Si WHATSAPP_PROVIDER=meta con secret: deploy normal.
- Si WHATSAPP_PROVIDER=meta sin secret: aborta con RuntimeError [META]. Revertir.

[] Si WHATSAPP_PROVIDER=meta activo: enviar un mensaje de prueba a Dona y verificar que llega. Verificar logs Render por '[META] Firma HMAC del webhook NO coincide'.

[] Si WHATSAPP_PROVIDER=whapi activo: nada operacional cambia. T0.10 sigue abierto.

12. Resumen ejecutivo

Aspecto	Detalle
Riesgo que cierra	P0 — inyeccion de mensajes WhatsApp falsos via /webhook cuando provider es Meta sin app_secret
Patron	Check en ProveedorMeta.__init__ + defensa en profundidad en _verificar_firma. Distinto al patron de Stripe (no es check al import del modulo).
Archivos	3 (1 codigo, 1 docs, 1 tests)
Tamano	~25-35 lineas netas

Aspecto	Detalle
Tests	1 actualizado + 4 nuevos
Env vars	META_APP_SECRET obligatorio en Render solo si WHATSAPP_PROVIDER=meta
Compatibilidad	Path permisivo se mantiene en development test. Tests actualizados, no rotos.
Riesgo de deploy	Bajo si la matriz WHATSAPP_PROVIDER <-> META_APP_SECRET esta consistente
Reversible	Si, redeploy del commit anterior
Hallazgo separado	T0.10 nuevo: whapi.py no valida firma de webhook. P0 abierto independiente.

Estado: no implementado todavia. Esperando OK del owner con confirmacion de:

- Valor actual de WHATSAPP_PROVIDER en Render.
- Si META_APP_SECRET ya esta seteado en Render (en caso de que el provider sea Meta).
- Si abro T0.10 (whapi sin firma) como tarea separada en el backlog para tratar despues.